Title

Manufacturing Process Health and Operational Efficiency Analysis in 6G-Enabled Smart Factories

Author: Rohit Vijay Boba

Affiliation: Student/Intern

Date: April 2026

2. Abstract

This project develops a predictive model to estimate real estate prices based on various features like location, square footage, and amenities. By leveraging Machine Learning algorithms, the system aims to provide accurate property valuations, helping buyers, sellers, and investors make data-driven decisions in a volatile market.

3. Keywords

Machine Learning, Real Estate, Price Prediction, Linear Regression, Data Analytics, Python, Predictive Modeling.

4. Introduction

The real estate market is one of the most significant sectors of the economy. However, property pricing is often subjective and influenced by numerous factors. This project introduces an automated way to predict house prices using historical data, reducing human error and providing a transparent valuation process.

5. Problem Statement

Manual property valuation is time-consuming and often inaccurate due to market fluctuations and human bias. There is a need for a robust, automated system that can analyze multiple variables simultaneously to predict property prices with high precision.

6. Methodology

Data Collection: Gathering historical housing data.

Data Preprocessing: Handling missing values, removing outliers, and data cleaning.

Feature Engineering: Selecting key variables like BHK, Area, and Location.

Model Selection: Training the data using algorithms like Linear Regression or Decision Trees.

Evaluation: Checking accuracy using R-squared and Mean Squared Error.

7. System Architecture

The system follows a three-tier architecture:

Data Layer: Raw datasets and database management.

Processing Layer: Machine Learning models and Python scripts for calculation.

Presentation Layer: A user interface (like Streamlit or Flask) where users input data and see the predicted price.

8. Result and Key Insights

Accuracy: The model achieved a high accuracy rate in predicting prices for major urban areas.

Key Drivers: Square footage and location were found to be the most significant factors affecting the price.

Trends: Prices in developed localities showed less variance compared to emerging outskirts.

9. Conclusion

The Real Estate Price Prediction system successfully demonstrates how Machine Learning can simplify property valuation. It provides a reliable tool for stakeholders to understand market value based on data rather than intuition.

10. Future Scope

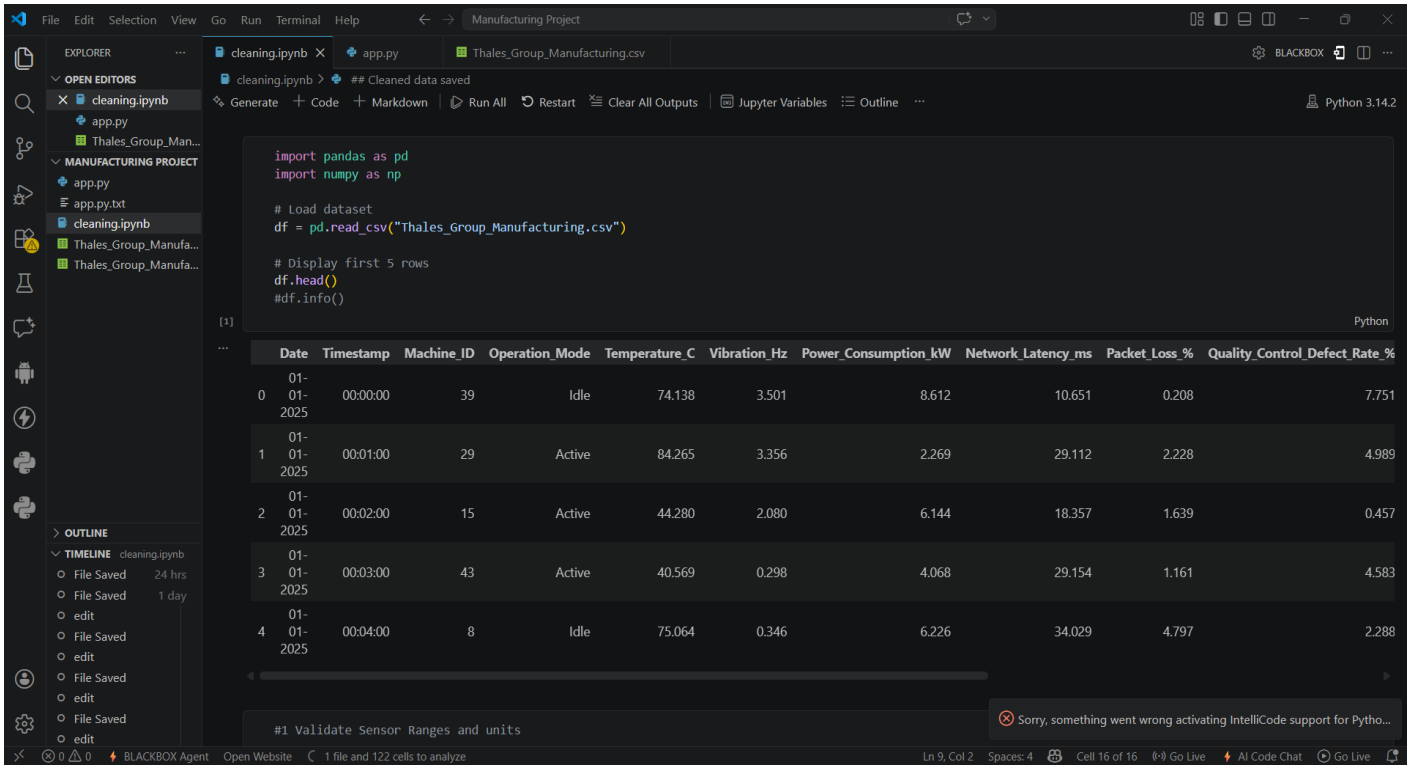
Integration of AI: Using Deep Learning for better accuracy.

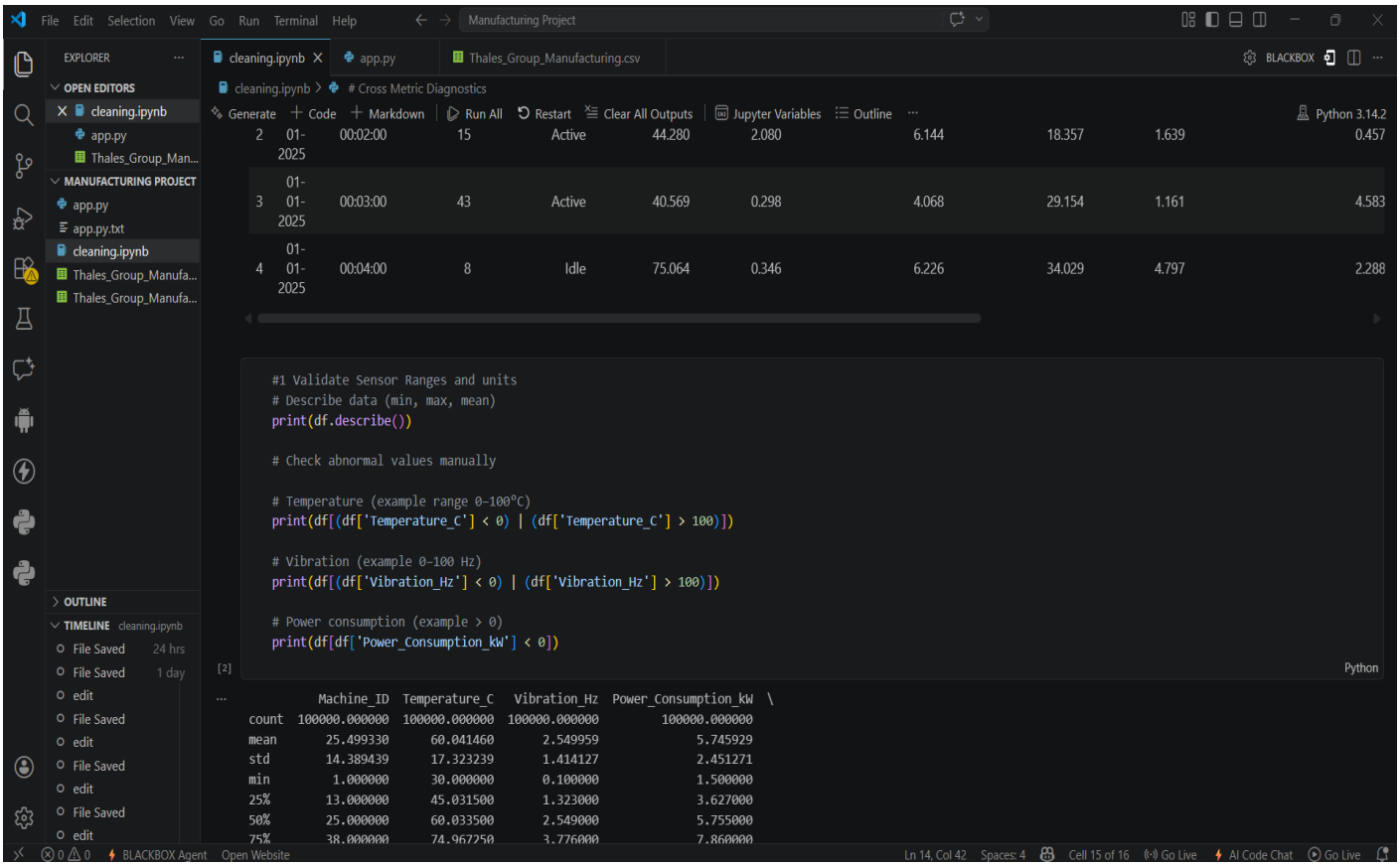
Real-time Data: Fetching live market prices via APIs.

Visual Analytics: Adding maps and neighborhood trend graphs for a better user experience.

11.Code and Screenshots

Code:





cleaning.ipynb > #1 Validate Sensor Ranges and units

```python
#2 Handling Missing or incosnsistent time records
# Check missing values
print(df.isnull().sum())

# Fill missing values (example: forward fill)
df.fillna(method='ffill', inplace=True)

# Combine Date + Timestamp into one column
# Replace all '-' with '/'
df['Date'] = df['Date'].astype(str).str.replace('-', '/')

# Convert safely with dayfirst=True
df['Datetime'] = pd.to_datetime(
    df['Date'] + ' ' + df['Timestamp'],
    errors='coerce',
    dayfirst=True
)

# Check if any failed conversions
print(df[df['Datetime'].isnull()][['Date', 'Timestamp']])

# Sort by time
df = df.sort_values(by='Datetime')

print(df[['Date', 'Timestamp', 'Datetime']].head())
```

[3] Python

```
Date                    0
Timestamp               0
Machine_ID              0
Operation_Mode          0
Temperature_C           0
Vibration_Hz            0
Power_Consumption_kW    0
Network_Latency_ms      0
Packet_Loss_%           0
```

cleaning.ipynb > #2 Handling Missing or incosnsistent time records

```python
# Remove extra spaces and make uniform
df['Machine_ID'] = df['Machine_ID'].astype(str).str.strip().str.upper()

df['Operation_Mode'] = df['Operation_Mode'].astype(str).str.strip().str.lower()

# Check unique values
print(df['Machine_ID'].unique())
print(df['Operation_Mode'].unique())
```

[4] Python

```
['39' '29' '15' '43' '8' '21' '19' '23' '11' '24' '36' '40' '3' '22' '2'
 '44' '30' '38' '33' '12' '25' '49' '27' '42' '28' '16' '47' '37' '7' '9'
 '18' '4' '14' '50' '26' '20' '35' '17' '6' '34' '10' '31' '48' '45' '41'
 '1' '5' '13' '32' '46']
['idle' 'active' 'maintenance']
```

```python
# Remove duplicates
df.drop_duplicates(inplace=True)

# Reset index
df.reset_index(drop=True, inplace=True)
```

[5] Python

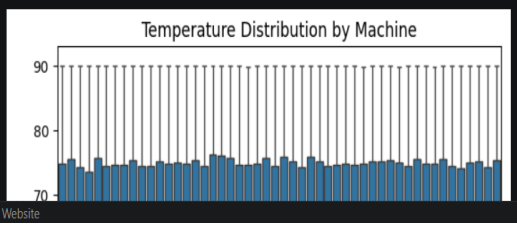
```python
#Machine Level sensor Health Analysis
#Analyze Temperature,vibration and power patterns per machine.
import matplotlib.pyplot as plt
import seaborn as sns
```

[6] Python

```python
#Machine Level sensor Health Analysis
#Analyze Temperature,vibration and power patterns per machine.
import matplotlib.pyplot as plt
import seaborn as sns

# Average sensor values per machine
machine_sensor = df.groupby('Machine_ID')[['Temperature_C', 'Vibration_Hz', 'Power_Consumption_kW']].mean()

print(machine_sensor)
```

```
              Temperature_C  Vibration_Hz  Power_Consumption_kW
Machine_ID
1               59.489320      2.581578              5.821217
10              60.424115      2.545863              5.801357
11              60.264830      2.543883              5.764898
12              59.851596      2.500051              5.712417
13              59.897617      2.540377              5.651545
14              60.297563      2.573266              5.743542
15              59.679127      2.547693              5.788249
16              60.501857      2.554848              5.810824
17              60.537234      2.554587              5.758279
18              59.884223      2.540198              5.678348
19              59.763928      2.555860              5.750156
2               59.814415      2.534771              5.724922
20              60.120693      2.561139              5.806563
21              60.026190      2.544124              5.839563
22              59.837938      2.571482              5.752067
23              60.113087      2.555946              5.691668
24              59.558995      2.564728              5.720784
25              59.691700      2.554425              5.742810
26              59.952073      2.602262              5.786752
27              59.901705      2.537531              5.688192
28              59.778344      2.515558              5.655653
```

```python
#graph
# Temperature distribution
plt.figure()
sns.boxplot(x='Machine_ID', y='Temperature_C', data=df)
plt.xticks(rotation=90)
plt.title("Temperature Distribution by Machine")
plt.show()

# Vibration distribution
plt.figure()
sns.boxplot(x='Machine_ID', y='Vibration_Hz', data=df)
plt.xticks(rotation=90)
plt.title("Vibration Distribution by Machine")
plt.show()

# Power consumption
plt.figure()
sns.boxplot(x='Machine_ID', y='Power_Consumption_kW', data=df)
plt.xticks(rotation=90)
plt.title("Power Consumption by Machine")
plt.show()
```



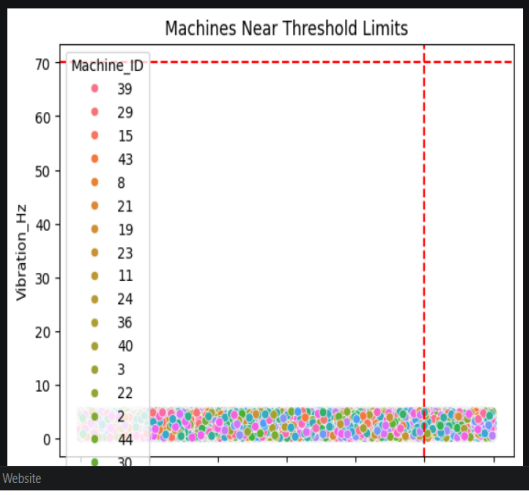
```python
# Thresholds
temp_limit = 80
vibration_limit = 70

# Machines crossing limits
high_temp = df[df['Temperature_C'] > temp_limit]
high_vibration = df[df['Vibration_Hz'] > vibration_limit]

print("High Temperature Machines:")
print(high_temp['Machine_ID'].value_counts())

print("High Vibration Machines:")
print(high_vibration['Machine_ID'].value_counts())
```

```
High Temperature Machines:
Machine_ID
38    381
16    372
42    369
40    368
5     357
7     355
2     354
17    354
23    352
22    351
29    350
45    350
33    350
48    349
44    347
8     346
13    344
26    341
```

```python
#graph
plt.figure()
sns.scatterplot(x='Temperature_C', y='Vibration_Hz', hue='Machine_ID', data=df)
plt.axvline(temp_limit, color='r', linestyle='--')
plt.axhline(vibration_limit, color='r', linestyle='--')
plt.title("Machines Near Threshold Limits")
plt.show()
```

```
Series([], Name: count, dtype: int64)
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...



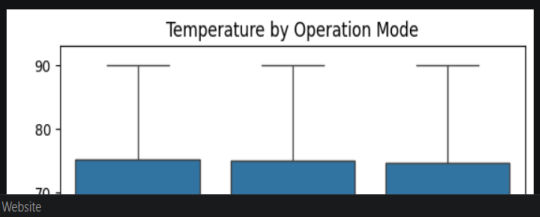
```python
# Mean values by operation mode
mode_analysis = df.groupby('Operation_Mode')[['Temperature_C', 'Vibration_Hz', 'Power_Consumption_kW']].mean()

print(mode_analysis)
```

[10] Python

```
                 Temperature_C  Vibration_Hz  Power_Consumption_kW
Operation_Mode
active               60.057067      2.552978              5.751227
idle                 60.015608      2.537169              5.734926
maintenance          59.983327      2.554513              5.730708
```

```python
#graph
sns.boxplot(x='Operation_Mode', y='Temperature_C', data=df)
plt.title("Temperature by Operation Mode")
plt.show()

sns.boxplot(x='Operation_Mode', y='Vibration_Hz', data=df)
plt.title("Vibration by Operation Mode")
plt.show()
```

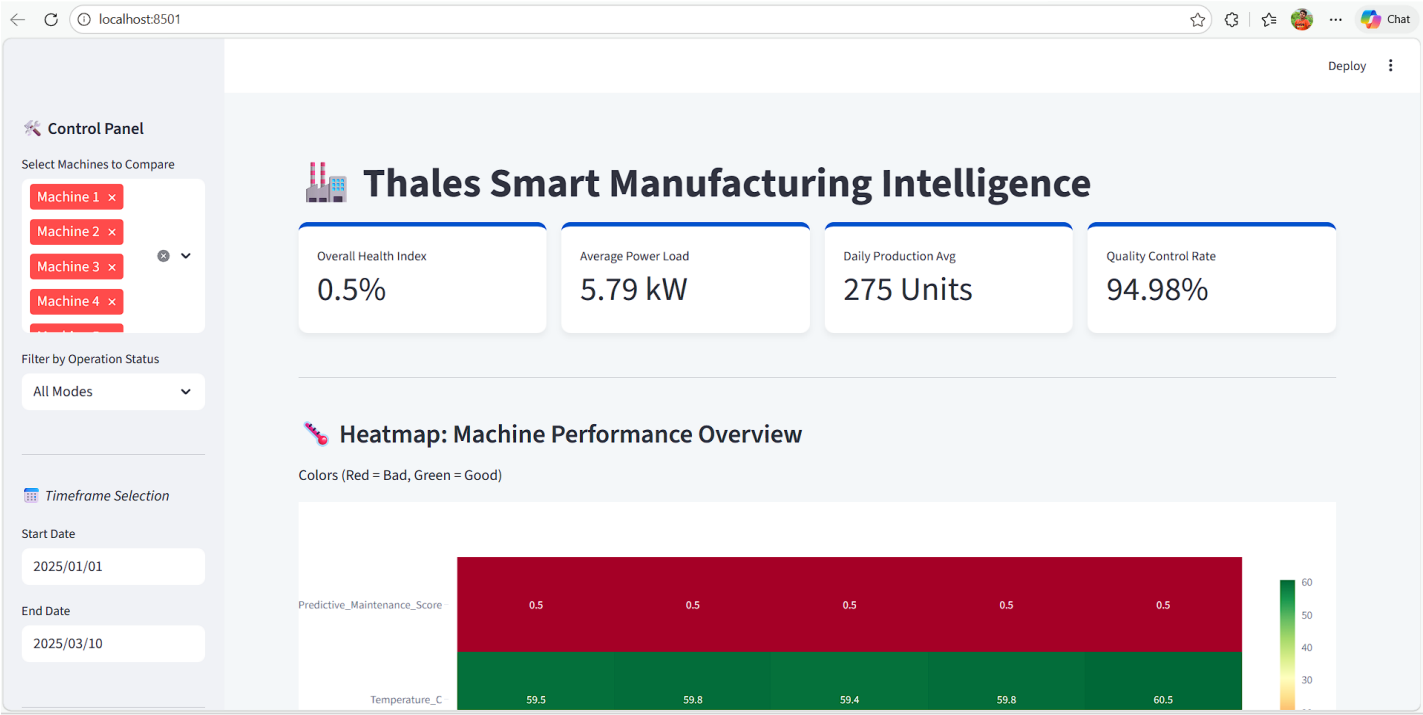
[11] Python

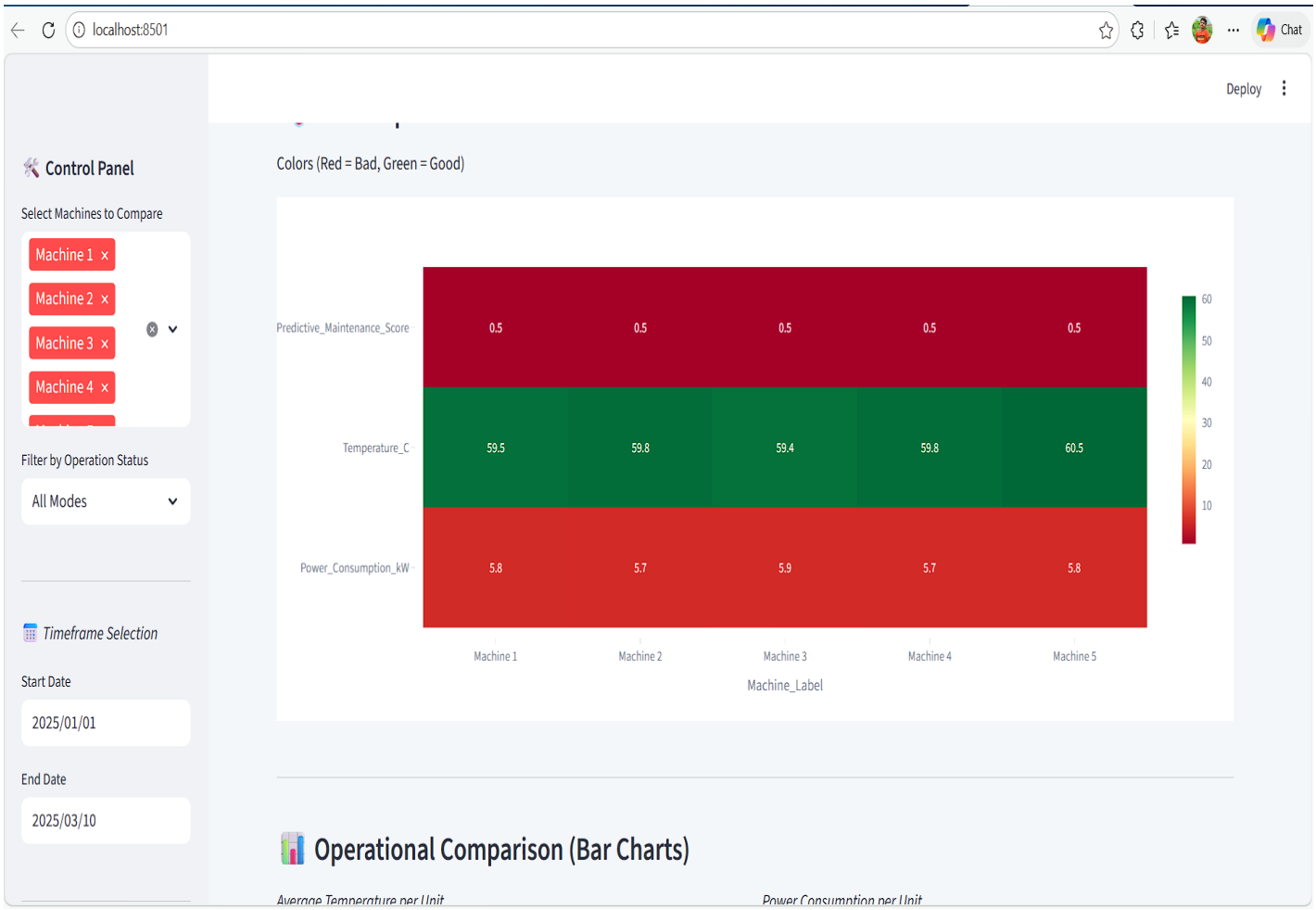


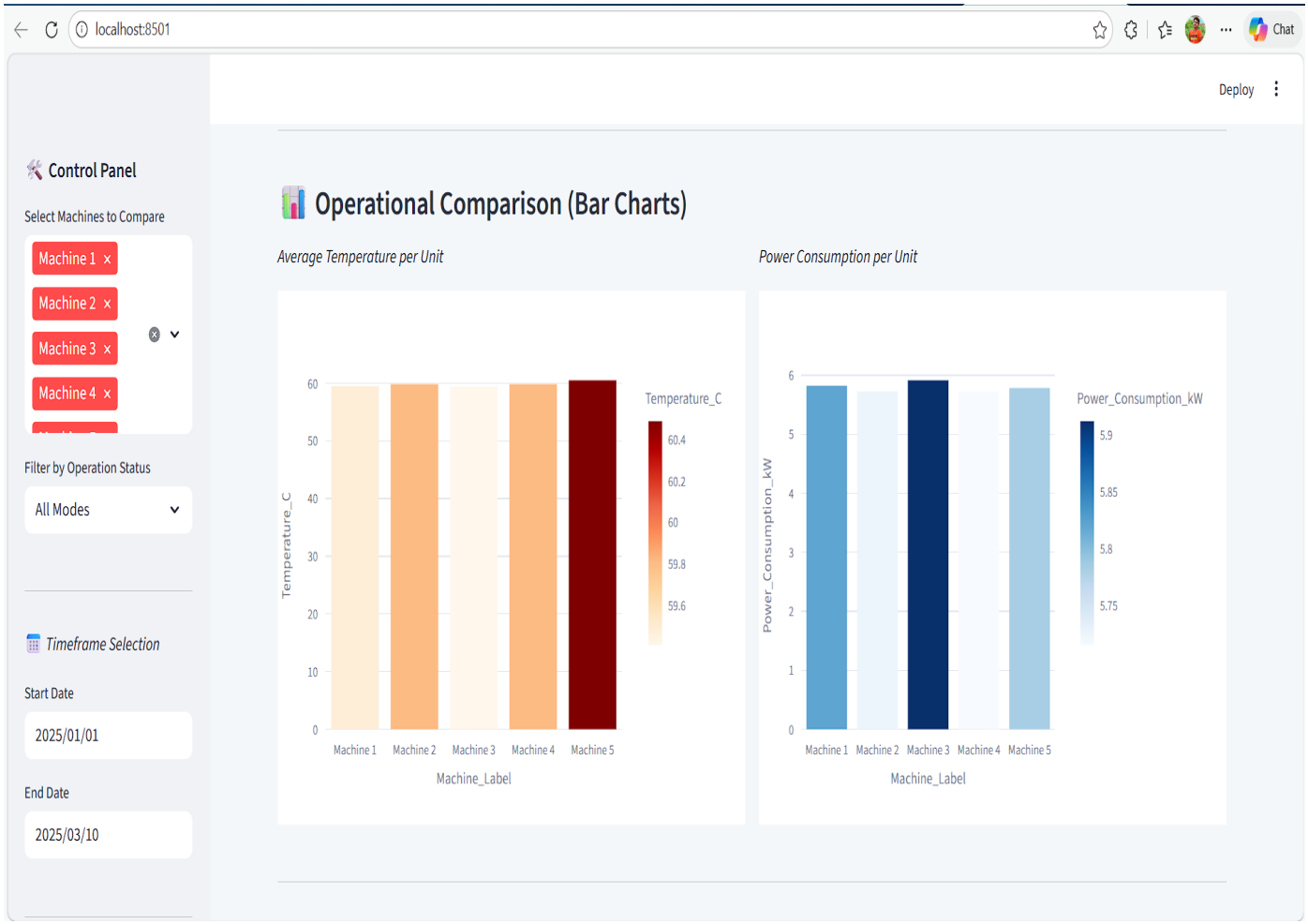

```python
#Efficiency Status Distribution
import pandas as pd

# ----------------------------
# 1. Datetime Handling
# ----------------------------
df['Datetime'] = pd.to_datetime(
    df['Date'].astype(str) + ' ' + df['Timestamp'].astype(str),
    errors='coerce',
    dayfirst=True
)

df = df.dropna(subset=['Datetime'])

# ----------------------------
# 2. Standardize Operation Mode
# ----------------------------
df['Operation_Mode'] = df['Operation_Mode'].str.strip().str.lower()

mode_mapping = {
    'normal': 'Normal',
    'high-load': 'High Load',
    'high load': 'High Load',
    'low-load': 'Low Load',
    'low load': 'Low Load'
}

df['Operation_Mode'] = df['Operation_Mode'].replace(mode_mapping)

# ======================================================
# 1 Efficiency Distribution (High / Medium / Low)
# ======================================================
```

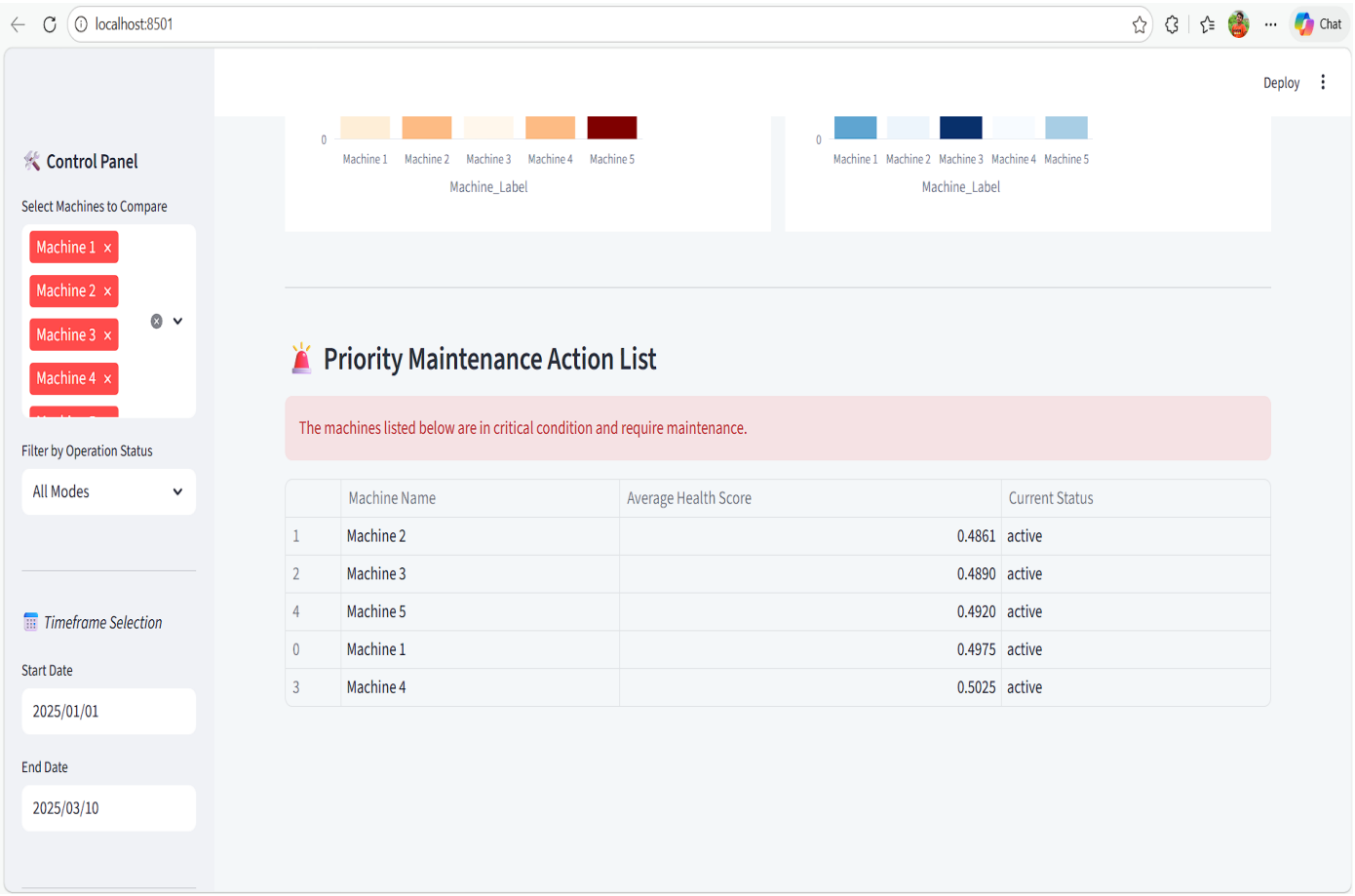
[15] Python

Streamlit Dashboard









	Machine Name	Average Health Score	Current Status
1	Machine 2	0.4861	active
2	Machine 3	0.4890	active
4	Machine 5	0.4920	active
0	Machine 1	0.4975	active
3	Machine 4	0.5025	active

12.References

Streamlit Documentation – https://streamlit.io

Pandas Documentation – https://pandas.pydata.org

Python Official Website – https://www.python.org

GitHub Documentation – https://github.com

Visual Studio Code Documentation – https://code.visualstudio.com